MetaEformer: Unveiling and Leveraging Meta-patterns for Complex and Dynamic Systems Load Forecasting

Shaoyuan Huang*
Tianjin University
Jinnan Qu, Tianjin Shi, China
hsy_23@tju.edu.cn

Tiancheng Zhang*
Tianjin University
Jinnan Qu, Tianjin Shi, China
tianchengzhang@tju.edu.cn

Zhongtian Zhang
Tianjin University
Jinnan Qu, Tianjin Shi, China
zzt_2022@tju.edu.cn

Xiaofei Wang†
Tianjin University
Jinnan Qu, Tianjin Shi, China
xiaofeiwang@tju.edu.cn

Lanjun Wang
Tianjin University
Jinnan Qu, Tianjin Shi, China
wang.lanjun@outlook.com

Xin Wang
Tianjin University
Jinnan Qu, Tianjin Shi, China
wangx@tju.edu.cn

arXiv:2506.12800v1 [cs.LG] 15 Jun 2025

Abstract

Time series forecasting is a critical and practical problem in many real-world applications, especially for industrial scenarios, where load forecasting underpins the intelligent operation of modern systems like clouds, power grids and traffic networks. However, the inherent complexity and dynamics of these systems present significant challenges. Despite advances in methods such as pattern recognition and anti-non-stationarity have led to performance gains, current methods fail to consistently ensure effectiveness across various system scenarios due to the intertwined issues of complex patterns, concept-drift, and few-shot problems. To address these challenges simultaneously, we introduce a novel scheme centered on fundamental waveform, a.k.a., meta-pattern. Specifically, we develop a unique Meta-pattern Pooling mechanism to purify and maintain meta-patterns, capturing the nuanced nature of system loads. Complementing this, the proposed Echo mechanism adaptively leverages the meta-patterns, enabling a flexible and precise pattern reconstruction. Our Meta-pattern Echo transformer (MetaEformer) seamlessly incorporates these mechanisms with the transformer-based predictor, offering end-to-end efficiency and interpretability of core processes. Demonstrating superior performance across eight benchmarks under three system scenarios, MetaEformer marks a significant advantage in accuracy, with a 37% relative improvement on fifteen state-of-the-art baselines.

CCS Concepts

• **Applied computing** → **Forecasting**.

Keywords

Time Series Forecasting, Industrial System, Meta-pattern

*Both authors contributed equally to this research.
†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/25/08
https://doi.org/10.1145/3711896.3737047

ACM Reference Format:
Shaoyuan Huang, Tiancheng Zhang, Zhongtian Zhang, Xiaofei Wang, Lanjun Wang, and Xin Wang. 2025. MetaEformer: Unveiling and Leveraging Meta-patterns for Complex and Dynamic Systems Load Forecasting. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3737047

KDD Availability Link:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.15502862.

1 Introduction

Time series forecasting, a fundamental task in machine learning, has long been the subject of extensive research across a multitude of domains such as the industry failure maintenance [5], medical diagnosis [16], and weather nowcasting [43]. This focus stems from its ability to predict relevant figures in advance, assisting in decision-making and formulating response strategies. As technology advances, an increasing number of modern systems (like cloud computing platforms [17, 18, 26], power grids [14, 39], and traffic networks [15, 20]) have integrated load forecasting capabilities into their operational processes. This integration marks a shift of systems from mere functionality to intelligent services.

However, these systems, which are characterized by **complexity** and **dynamics**, introduce significant challenges for load forecasting [35]. Specifically, the system **complexity** arises from its diverse internal entities and external influences like natural, economic, and social factors. For instance, the multitude of servers and applications in cloud platforms; similarly, in traffic systems, it manifests through the varying characteristics of different roads and traffic patterns. The confluence of complexity exhibits intricate patterns with seasonality, trend, outlier, and noise, which tests the latent pattern recognition capabilities of predictors [35].

On the other hand, real-world systems exhibit extremely high dynamics, a characteristic stemming from the evolving state of systems entities as they operate and interact with the external environment, which can be attributed to two types of entity behaviors:

(i) Entity transition and switching. As illustrated in Fig. 1, frequent application switches in cloud platforms could cause load pattern changes, similar to seasonal migration in power grids and

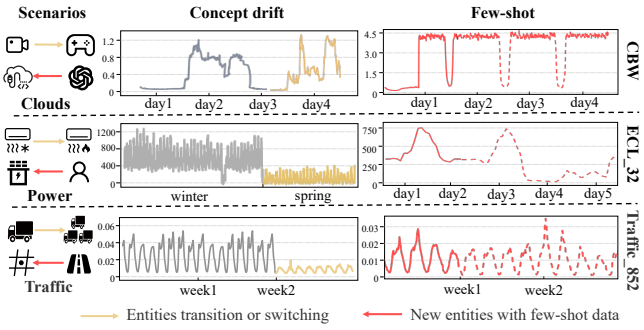


Figure 1: Systems loads variations due to entity behaviors.

traffic congestion transitions in road networks. In time series forecasting, the phenomenon of time-variant data distribution is identified as **concept-drift** [31]. In this context, models often experience performance degradation due to their focus on outdated patterns, failing to promptly capture the post-switch patterns [2, 13, 22].

(ii) Introduction of new entities. As Fig. 1 shows, newly added entities may bring unknown load patterns. This could be new types of application in cloud platforms, newly connected industrial users in power grids, or new segments in traffic networks. In such scenarios, models are challenged to capture new patterns from the limited data of the **few-shot** [11], new entities, testing their ability to generalize and reconstruct new patterns from existing knowledge.

Recent research in time series forecasting has witnessed significant strides, with some methods addressing certain challenges in system load forecasting: generic deep learning models excel at pattern recognition, capturing complex load patterns, while *decomposition-based methods* [9, 38, 46] further enhance this by directly utilizing latent pattern information. *Anti-non-stationary approaches* [28–30] demonstrate robustness to load variability, actively adapting to dynamic factors like frequent entity transitions. *Clustering-based strategies* [10, 19, 39], commonly used in industrial settings, leverage pre-constructed model pools to extract targeted load patterns and adapt dynamically through model switching.

Despite these efforts, existing methods each exhibits inherent limitations when attempting to tackle the challenges of complexity, concept drift, and few-shot scenarios simultaneously. Furthermore, the lack of interpretability remains a key concern in real-world industrial contexts, especially within dynamic systems.

This paper's goal is to concurrently address the complexity and dynamics challenges of systems load forecasting, while providing the efficiency and interpretability required for real-world systems. Inspired by Wavelet Transform principles [33], we suggest that all load patterns and their variations in a system are composed of fundamental waveforms. To achieve this, such waveform should exhibit fine granularity and be highly representative, going beyond traditional periodic patterns to form a 'pattern within patterns', a.k.a., meta-pattern. Based on this idea, we design the ***Meta-pattern Pooling*** and a novel ***Echo mechanism*** to enhance the transformer-based predictors' focus on the essential differences and connections in complex and dynamic load patterns.

The Meta-pattern Pooling design draws inspiration from wavelet basis, which are fundamental elements that exhibit temporal locality and form all vectors in a space. The pooling mechanism integrates time series decomposition and threshold-based pattern purification to isolate these basic units (meta-patterns) from load series and pool them into the meta-pattern pool (MPP) for maintenance.

The Echo mechanism is designed to deconstruct the target load series and reconstruct load patterns, the concept is akin to the process in wavelet transform where the wavelet basis is used to analyze the original signal. Specifically, the Echo enhances complex pattern recognition by integrating relevant meta-patterns. For concept-drift problems, it identifies post-transition patterns from the MPP, mitigating the model's reliance on outdated information. For few-shot issues, the Echo combines existing meta-patterns in the pool to reconstruct unknown patterns, enabling rapid adaptation to unknown data distributions.

Both mechanisms are implemented as learnable layers with adaptive parameters, allowing them to be directly integrated into the encoder-decoder architecture as evolving companion modules. This avoids pre-built structures, prevents information leakage, and ensures long-term effectiveness. Our framework, Meta-pattern Echo transformer (**MetaEformer**), excels in various system load forecasting tasks, delivering seamless end-to-end processing and interpretability to meet industrial demands.

To summarize, our main contributions are as follows:

- We introduce a new perspective on meta-patterns and propose a novel *Meta-pattern Pooling* mechanism for purifying and maintaining these fundamental waveforms in system loads. This mechanism can autonomously evolve alongside the model training and inference, capturing the intricacies of load series.
- To leverage the meta-patterns, we propose an innovative *Echo* mechanism that deconstructs load series and reconstructs patterns. This mechanism simultaneously adapts to load forecasting challenges like complex pattern, concept-drift, and few-shot problems, offering a more flexible and precise forecasting model.
- Our framework, MetaEformer, integrates these mechanisms as learnable and adaptive components into the transformer-based predictor. This design not only achieves superior performance on eight real-world system load datasets, but also offers end-to-end efficiency and clear interpretability of core processes.

2 Related Work

In this section, we review previous research regarding the aforementioned decomposition-based, anti-non-stationary and clustering-based approaches.

Decomposition-based methods integrate time series decomposition with advanced predictors to explicitly identify patterns by extracting the key components (such as the seasonal component) from the original time series. Wu et al. [38] introduces traditional decomposition method [4] and auto-correlation into the transformer for superior complex long-term forecasting. Following this paradigm, other works also integrate various decomposition techniques. Examples include the frequency-based FEDformer [46] and DLinear [42], and the period and phase-focused Quatformer [9].

These methods are evaluated as effective in extracting predictive information from complex time series (especially for system loads in their experiments). However, from a dynamism perspective, their effectiveness diminishes, especially when facing changing or

Table 1: Comparison of different types of methods, focusing on advantages applicable to complex and dynamic series forecasting.

Methods	End-to-End	Pattern Recognition	Dynamics Adaptation	Interpretability
Generic Models [7, 32, 40, 41, 44]	✓	✓	✗	✗
Decomposition-based [9, 38, 42, 46]	✓	✓	✗	✗
Anti-non-stationarity [28–30]	✓	✗	✓	✗
Clustering-based [10, 17, 19, 21, 39]	✗	✓	✓	✓
Ours	✓	✓	✓	✓

unknown load patterns, since over-reliance on historical patterns could misleadingly direct the model.

Anti-non-stationary approaches are emerging techniques that focus on analyzing loads' time-variant characteristics and developing methods to eliminate and reconstruct non-stationary aspects, enhancing models' resilience in dynamic scenarios.

Liu et al. [29] propose a "stabilize then destabilize" method, addressing the limitations of traditional preprocessing like first-order differencing in Transformers, using a destabilizing attention mechanism for better adaptation to non-stationary data. Similarly, Liu et al. [30] note the potential oversight of varying statistical features by stabilization techniques, advocating for a segment-based approach with a lightweight network to handle non-stationarity series.

While these methods refine traditional stabilization preprocessing methods, they still deconstruct the inputs directly and risk losing key non-stationarity details crucial for real-world systems. Besides, these methods do not provide a clear interpretability of how the system dynamics are represented and perceived.

Clustering-based strategies enhance model robustness by maintaining multiple data and model pools that jointly contribute to forecasting. This kind of paradigm is commonly employed for system-oriented forecasting tasks. In the cloud platforms, both Kim et al. [21] and Jayakumar et al. [19] introduce multiple model-based predictions using clusters of computational loads. Similarly, in the power load domain, concepts akin to data pooling have been implemented in work [39].

The clustering-based strategies yield two primary advantages. First, they facilitate statistical pattern recognition through data clusters, thereby enhancing comprehension of system load patterns. Second, they adeptly interpret dynamic behaviors, such as entity transitions and the integration of new entities, through model switching within the framework [10]. However, the primary drawback of these methods is their complexity. Discarding the end-to-end pipeline for multiple data clusters and models escalates computational expenses. Despite Huang et al. [17] making efforts to incorporate the capabilities of multiple models into a single framework, it is still not completely end-to-end. Their reliance on full historical data for global pools raises concerns about potential information leakage.

In summary, as shown in Table 1, leveraging the innovate meta-patterns, MetaEformer harnesses the core strengths of various approaches while overcoming their inherent limitations, making it particularly well-suited for system load forecasting tasks.

3 Notations and Problem Definition

System load forecasting. A load series comprises a sequence of load points (e.g., cloud server bandwidth, electricity consumption or traffic flow) generated over time t, denoted as follows:

$$\mathcal{X} = \{x_{t_1}, x_{t_2}, \ldots, x_{t_n} \mid x_t \in \mathbb{R}^d\} \tag{1}$$

where d is the feature dimension with $d \geq 1$. These data points typically appear at fixed time intervals k, and given a look-back window of size L, the objective is to predict the data for the next L_y intervals. Therefore, for a forecasting task starting from t, the input can be represented as $\mathcal{X}_t = (x_i)_{i=t-L*k}^{t}$, and the target is to predict the future segment $\mathcal{Y}_t = (x_i)_{i=t+k}^{t+L_y*k}$.

Concept-drift and Few-shot. Traditional time series forecasting assumes that x_i in $\mathcal{X}_t$ and $\mathcal{Y}_t$ follow the same distribution $\mathcal{D}_t$, i.e., $x_i \sim \mathcal{D}_t$. Besides, historical data distributions D are presumed to encompass any potential future distributions. However, under concept-drift, x_i may follow multiple distributions, for instance, $(x_i)_{i=t-L*k}^{S1} \sim \mathcal{D}_t^1$ and $(x_i)_{i=S1}^{S2} \sim \mathcal{D}_t^2$, and $S1$ is the time for entity transition. On the other hand, in few-shot scenarios, a future distribution $\mathcal{D}_t^{'}$ may not belong to any distribution learned by the model, signified as $\mathcal{D}_t^{'} \notin D$. Both scenarios significantly increase the difficulty of forecasting.

4 Methodology

In this section, we introduce the overall architecture of MetaEformer, and dive into the proposed Meta-pattern Pooling and Echo mechanism. The two mechanisms are designed as inherently interpretable components and integrated into the encoder-decoder architecture, ensuring an end-to-end pipeline while handling the complexity and dynamics caused by systems entities.

As illustrated in Fig. 2, MetaEformer sustains two main processes: Meta-pattern Pooling and Forecasting. The Meta-pattern Pooling involves extracting key components from load series and purifying meta-patterns, alongside maintaining the meta-pattern pool (MPP). The Forecasting elucidates how the proposed Echo Layer and Echo Padding incorporate the MPP to guide the predictive process of the encoder-decoder model. Notably, the two processes are not independent, but tightly coupled. In each computational round, the same input sequences are utilized both for forecasting and for purifying meta-patterns. The updated MPP is then fed into the Forecasting framework's Echo Layer, ensuring seamless integration and continuous refinement.

4.1 Meta-pattern Pooling

As aforementioned, meta-patterns can be descriptively defined as *fundamental waveform units that capture the essential periodic and structural components of complex and dynamic time series data.* Beyond this, we emphasize that the operational definition, i.e., *how*

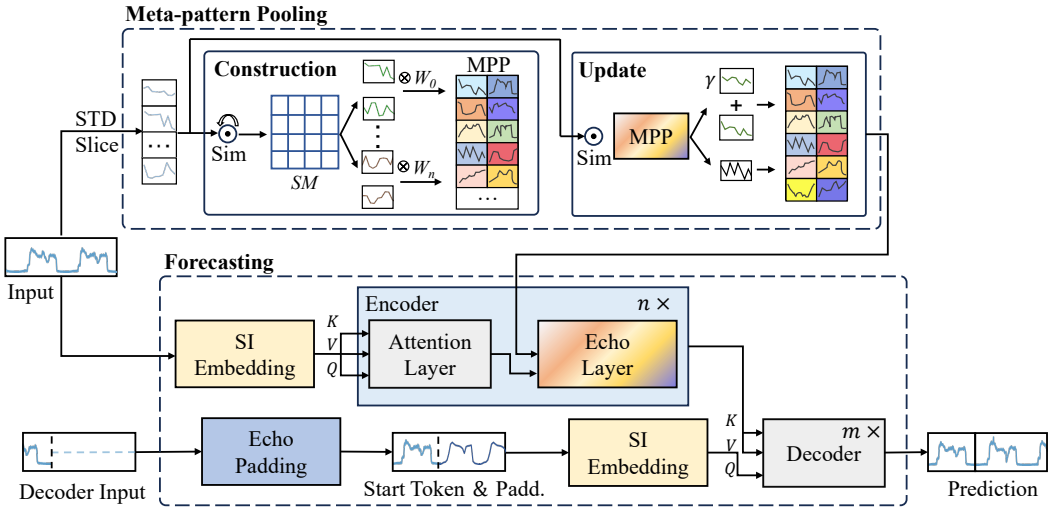


Figure 2: Framework overview of MetaEformer, consisting of two main parts: Meta-pattern Pooling (§4.1) and Forecasting (§4.2).

meta-patterns are purified from general patterns, is of greater practical value. This involves identifying finer-grained patterns and merging similar ones to achieve more representative meta-patterns.

4.1.1 ***Seasonal Component Extraction.*** The direct manifestation of the complexity and dynamics of system loads lies in their periodic patterns, and thus extracting the periodic patterns serves as the precursor to purifying the meta-patterns. To achieve this, we introduce decomposition [38] and seamlessly integrate it as a built-in component. This approach decomposes the load series into trend and seasonal components. Specifically, we use moving averages to smooth the input load fluctuation, treating it as the trend of the load. The process is as follows:

$$\begin{aligned} \mathcal{X}^T &= \text{AvgPool}(\text{Padding}(\mathcal{X})),\ \mathcal{X} \in \mathbb{R}^{L\times d} \\ \mathcal{X}^S &= \mathcal{X} - \mathcal{X}^T \end{aligned} \tag{2}$$

where $\mathcal{X}^T$, $\mathcal{X}^S \in \mathbb{R}^{L\times d}$ denote the trend and the seasonal part respectively. We adopt the AvgPool(·) for moving average with the padding operation to keep the series length unchanged. Subsequently, we use STD(·) to describe the whole decompose process.

The trend component indicates the underlying pattern in the data, while the seasonal component captures the periodic cycles. In real-world systems, the trend component is usually simple and stochastic due to human or other interference [36]. By employing decomposition, our aim is to extract components with greater representational and reconstructive potential. Therefore, the seasonal part is selected for purifying meta-patterns.

4.1.2 ***Meta-pattern Pool Construction.*** Although decomposition helps extract periodic patterns of load series, these isolated patterns are not yet quintessential meta-patterns. To refine them into distinct, highly representative meta-patterns, further purification is needed. MetaEformer achieves this by maintaining a meticulously designed MPP, involving construction and subsequent updates.

As illustrated in Fig. 2, the MPP is initialized with the **first training batch**, and the whole construction process is formalized in Algorithm 1. Specifically, the MPP ($\mathcal{P}$) with dimensions $P \times s$ (where the MPP size P is scenario-dependent and discussed in the experiment section) is initially filled with a masking value of zero. Subsequently, the initial batch (with batch size B) of time series data X undergoes STD(·) to extract the periodic patterns of loads.

In addition to STD(·), inspired by the patch operation in the image domain [6], we introduce the Slice(·) operation to yield a more fine-grained waveform matrix $W \in \mathbb{R}^{(BL/s)\times s}$ (where s is the waveform length). While a single time series may contain multiple waveforms, especially in complex and dynamic scenarios, as illustrated in Fig. 3, considering a time series with entity switching might encompass three or more patterns. Treating the entire sequence as a whole could lead to the loss of these patterns.

After extracting the waveform matrix, a crucial step in MPP construction is to measure the similarity between patterns and determine which patterns should be merged into a single meta-pattern. This process should adhere to the following principles:

1) Similar patterns should be maximally identified and merged to reduce redundant storage and computation.

2) Heterogeneous patterns should be maximally separated to fully exploit information and adapt complex dynamic scenarios.

Additionally, as a computational unit within the neural network, this process should be simple and efficient to avoid excessive computational complexity. Therefore, methods with warping-based similarity calculations, such as DTW (with complexity $O(s^2)$), and high-complexity clustering methods, are not suitable.

Waveform Similarity Measurement. Inspired by vector dot product similarity, we develop an alignment enhancement method $\text{Sim}(\cdot,\cdot)$ for measuring pattern similarity. Specifically, we approach from a waveform perspective, determining the similarity between patterns by aligning their fluctuations and augmenting the connection among similar patterns. The similarity between two waveforms $\mathcal{W}_1$ and $\mathcal{W}_2$ in the waveform matrix W is computed as follows:

$$\text{Sim}(\mathcal{W}_1, \mathcal{W}_2) = \sum_{i=1}^{n} w_{1i} \cdot w_{2i} \tag{3}$$

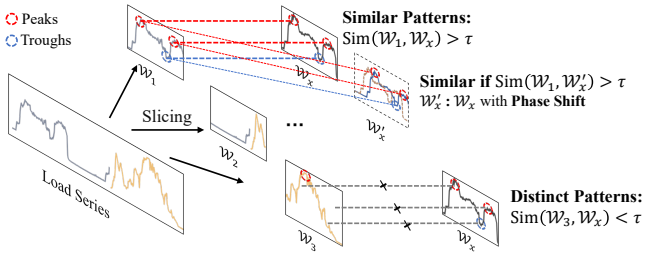


Figure 3: Waveform similarity measurement.

where w_{xi} denote the i-th value of $\mathcal{W}_x$. This method suggests that congruent peaks will lead to a higher Sim, facilitating pattern matching, while disparate patterns result in lower Sim due to peak and trough misalignment.

Sim enhances interpretability and is computationally efficient, making it ideal for end-to-end models. Moreover, due to the incorporating of standardization, the method mitigates amplitude variations, ensuring the similarity measurement focuses on the shape of waveforms rather than absolute high values.

After performing Sim between rows of W, we obtain the waveform similarity matrix SM (SM is structured as an upper triangular matrix to avoid redundant computations) as follows:

$$SM_{ij} = \begin{cases} \text{Sim}(W_i, W_j) & \text{for } i < j, \\ 0 & \text{for } i \geq j. \end{cases} \tag{4}$$

For time series forecasting task, *phase shifts* ($\mathcal{W}'_x$ in Fig. 3) contain critical predictive information. Therefore, the alignment based Sim retains the impact of phase shifts as part of the meta-pattern features, rather than eliminating it. Significant phase shifts with low Sim values are considered different meta-patterns, while slight phase shifts are considered the same meta-pattern. This distinction is achieved through the purification operation described below.

Threshold-based Purification. Sim measures waveform similarity but does not determine aggregation, making it insufficient to satisfy the principles of pattern clustering and independence. To address this, we introduce a threshold-based purification method. Given the purification threshold τ, patterns with Sim greater than τ are treated as identical meta-pattern, while others remain separate. Using the similarity matrix SM, we compute the purification threshold τ as follows:

$$\tau = \mu(SM) + \frac{\alpha P}{|SM|}\sigma(SM) \tag{5}$$

where $\mu(SM)$ represents the mean of the non-diagonal elements of SM, $\sigma(SM)$ denotes the standard deviation. Given the generally larger P, $\sigma(SM)$ would significantly impacts τ. To balance this, we scale $\frac{P}{|SM|}$ by a constant factor α (set to 0.5). This rationale behind fixing α while adjusting P aligns with the scenario's complexity, as demonstrated in §5.4.

The formula for τ is dynamically fluctuate with the MPP size P, which is scenario-dependent. More complex system loads require a larger P, resulting in a higher τ and ensuring more distinct patterns are recognized as separate meta-patterns, thus preventing excessive amalgamation. As outlined in Algorithm 1, after specifying SM and τ, MetaEformer selects all similar patterns with Sim value greater than τ for each pattern (line 8), fusing them into a meta-pattern weighted by their similarity (line 9) and populating the MPP (line 10), until the first batch of data is traversed.

Algorithm 1 Meta-pattern Pool Construction

Input: the first batch input time series X, the waveform length s, the MPP size P
Output: the meta-pattern pool $\mathcal{P}$

1: Initialize empty meta-pattern pool $\mathcal{P} \in \mathbb{R}^{P\times s}$ with mask 0
2: Extract the seasonal part of X and slice into waveform matrix $W = \text{Slice}(\text{STD}(X)), W \in \mathbb{R}^{(BL/s)\times s}$
3: /* The calculations are described in Eq. 4 and 5 */
4: Calculate similarity matrix SM and similarity threshold τ
5: **for** $i = 1$ to $|SM|$ **do**
6: Determine max similarity $v = \max(SM_i)$
7: **if** $v > \tau$ **then**
8: Identify similar waveforms $\mathcal{K} = \{k \mid SM_{ik} > \tau\}$
9: Purify weighted pattern $M = (W_{\mathcal{K}}^T \cdot \frac{SM_{i\mathcal{K}}}{\sum(SM_{i\mathcal{K}})})$
10: Append to MPP $\mathcal{P} \leftarrow M$
11: **else**
12: Append original waveform to MPP $\mathcal{P} \leftarrow W_i$
13: **end if**
14: **end for**
15: **return** $\mathcal{P}$

4.1.3 *Meta-pattern Pool Update.*

The initially constructed MPP, relying solely on the first batch of training data, is inherently limited in its capacity to encompass the various future load series. To address this limitation, we propose an MPP update mechanism (unfolded in Appendix A.1, Algorithm 2), enabling MPP to iteratively evolve and incorporate new data post-initialization. This mechanism ensures that the MPP dynamically adapts, extending its coverage to accommodate the evolving landscape of load series.

The mechanism occurs in two ways: 1) Distinct, novel waveforms are categorized as new meta-patterns in the MPP until its size P is reached. 2) Resembling waveforms from new data are merged into the most similar meta-patterns, enhancing their generality.

Notably, the Meta-pattern pooling mechanism can be regarded as an efficient and integrated clustering process, supporting the identification of similar load patterns and of unique load behaviors. Clustering-based approaches are widely recognized for their interpretability, as they organize data into meaningful structures [17, 39]. *The proposed mechanism inherits this property while avoiding the high computational complexity and potential information leakage.*

4.2 Forecasting Framework

The MetaEformer's forecasting framework is built on transformer-based predictor, primarily consists of multiple encoders and decoders, supplemented by the Static Inject (SI) Embedding module, Echo Layer and Echo Padding module, as shown in Fig. 2. The inputs for Forecasting framework include the time series data (with temporal marks), static context information related to the series generation, and MPP. MetaEformer utilizes a start token and serves as a direct multi-step (DMS) forecasting, avoiding the error accumulation associated with the iterated multi-step (IMS) approach [3].

4.2.1 ***SI Embedding.*** In real-world systems, the load varies with the environment [27]. Taking edge cloud applications for instance, where devices exhibit significant heterogeneity. The hardware environment in which applications are deployed can influence their load patterns. Therefore, going beyond the classical Transformer's Position Embedding [34] and the Informer's Temporal Embedding [45], we design the SI Embedding to inject additional static information.

Differing from the traditional approach that simply stacking static context onto model inputs [27], SI Embedding autonomously merges the temporal and static information through two adaptive linear layers. Consequently, the model can learn the relationship between static information and load patterns in a high-dimensional space, enhancing robustness in different external environments and contributing to overcoming challenges posed by entity changes and new entities. The computation of SI Embedding is as follows:

$$\begin{aligned} E^{SI} &= \text{Linear}_1(\mathcal{S}) \\ E^{X} &= \text{Token}(X) + \text{Temporal}(X) + \text{Position}(X) \\ E^{F} &= \text{Softmax}(\text{Linear}_2(E^{X})) \cdot E^{SI} \\ E^{X} &= \text{LayerNorm}(E^{X} + \text{Dropout}(E^{F})) \end{aligned} \tag{6}$$

where $\mathcal{S}$ represents static context and X represents the input time series. Token, Temporal, Position respectively signify TokenEmbedding, TemporalEmbedding and PositionEmbedding. E^F denotes the final fused Embedding.

4.2.2 ***Echo Layer.*** The encoder of MetaEformer consists of three parts: the full Attention Layer, Meta-pattern Pool ($\mathcal{P}$) and the Echo Layer, which is formalized as follows:

$$\mathcal{P} = \text{Meta-pattern Pooling}(X) \tag{7}$$

$$A^{en} = \text{Attention}(E^{F}) \tag{8}$$

$$O^{en} = \text{Echo Layer}\left(A^{en}, \mathcal{P}\right) \tag{9}$$

where Meta-pattern Pooling can be the MPP construction when X is the first batch of input series or the MPP update when X are subsequent inputs. A^{en} is the result of the Attention Layer, and O^{en} is the output of the encoder.

As the core of MetaEformer, the Echo Layer (as shown in Fig. 4) is designed to enhance encoder's capabilities by echoing back the most similar meta-patterns, thus focusing the Attention function on the inherent connections and differences in patterns. Specifically, the Echo Layer implements two main operations: deconstructing load series and reconstructing load patterns. The deconstructing operation is to identify the input load series into Top-K most similar meta-patterns by leveraging MPP. The detailed process is as follows:

$$W_1, \cdots, W_n = \text{Slice}_{(2)}(A^{en}[:, :, \frac{1}{2}d_{model}:]), n = L/s \tag{10}$$

$$\mathcal{P}^{i}_{select} = \text{Top-K}\left(\text{Sim}(\text{Linear}(W_i), \mathcal{P})\right) \tag{11}$$

where $\text{Slice}_{(2)}(\cdot)$ denotes the Slice in the second temporal-dimension of A^{en}, $W_i \in \mathbb{R}^{B\times s\times \frac{1}{2}d_{model}}$ is the waveform obtained by slicing from A^{en}. After that, each waveform W_i is used to Sim with the MPP to identify the K most similar patterns, denoted as $\mathcal{P}^{i}_{select} \in \mathbb{R}^{K\times s}$, Linear represents a dimension reduction operation in order to align the dimensions of W_i with $\mathcal{P}$. Note that we use the last $\frac{1}{2}$ features of A as the operation object, while the first $\frac{1}{2}$ features are retained for keeping the original information.

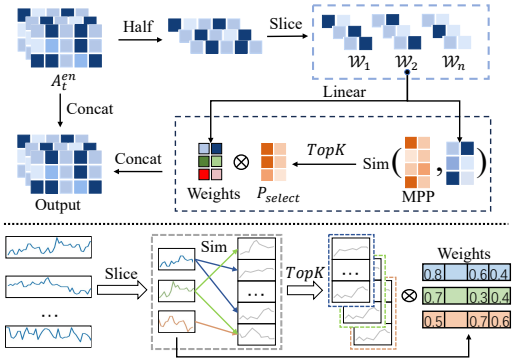


Figure 4: Echo Layer.

The reconstructing is to reintegrate the identified meta-patterns as new features in load series, which can be formalized as follows:

$$O_i = \text{Softmax}\left(DR(W_i)\right) \cdot (\mathcal{P}^{i}_{select})^{T} \in \mathbb{R}^{B\times s\times K} \tag{12}$$

$$\mathbf{O} = \text{Concat}\left(DI(O_1, \cdots, O_n)\right) \in \mathbb{R}^{B\times L\times \frac{1}{2}d_{model}} \tag{13}$$

$$O^{en} = \text{Concat}(A^{en}[:, :, :\frac{1}{2}d_{model}], \mathbf{O}) \in \mathbb{R}^{B\times L\times d_{model}} \tag{14}$$

where DR is the dimension reduction Linear to align W_i with $\mathcal{P}^{i}_{select}$, DI is the dimension increase Linear to recovery the needed dimension. In Eq. 12, the Echo utilizes W_i to adaptively generate the weights to merge $\mathcal{P}^{i}_{select}$, extracting information from the Top-K similar meta-patterns. After that, K will be the new feature dimension. The information merged from multiple meta-patterns is then concatenated with the first half of A^{en}, forming the final output.

4.2.3 ***Decoder.*** The decoder comprises Echo Padding, SI Embedding, and a stack of M decoder layers. Assuming the latent variables O^{en} as the output of the encoder and X^{de} as the decoder input, the decoder can be formalized as follows:

$$\begin{aligned} X^{de} &= \text{Echo Padding}(X^{de}) \\ E^{de} &= \text{SI Embedding}(X^{de}) \\ O^{de} &= \text{decoder}(E^{de}) \end{aligned} \tag{15}$$

4.2.4 ***Echo Padding.*** Decoder inputs padding is a key component for transformer-based models to achieve DMS prediction [45]. The previous practice use mean or simply zeros placeholders to fill the prediction length, lack interpretability and fail to provide additional information, thus limiting model's performance. To address this, we propose the Echo Padding to reuse the powerful MPP and fuse the relevant meta-patterns for filling the decoder inputs:

$$X^{EP} = \text{Contact}^{n}_{i=1}\left(\text{Softmax}\left(DR(W_i)\right) \odot (\mathcal{P}^{i}_{select})^{T}\right) \in \mathbb{R}^{B\times L_y\times d}$$

$$X^{de} = \text{Concat}\left(X[:, L_{token}:, :], X^{EP}\right) \in \mathbb{R}^{B\times (L_{token}+L_y)\times d}$$

where $\odot$ is the matrix multiplication form of Eq. 12, L_{token} is the token length. This design allows the decoder to reconstruct the predictive series based on meta-patterns, enhancing the model's information utilization and predictive performance.

4.3 Negligible Additional Complexity

While the Meta-pattern Pooling and Echo mechanisms these mechanisms add components to the time series Transformer, MetaEformer carefully manages the construction and updating of the MPP, and the Echo Layer selectively employs the Top-K meta-patterns, thus keeping the additional computational complexity relatively low.

We analyze the complexity of MetaEformer and compare it with that of the traditional time series Transformer (detailed in the Appendix A.2). In summary, due to MetaEformer's careful computation management, the complexity of additional mechanisms could be hundreds of times ($\frac{N(Ld+d^2)}{K}$) **lower** compared to the original attention and feed-forward computations, rendering it **negligible**.

Additionally, it is noteworthy that MetaEformer remains competitive in computational efficiency, even with full-attention mechanism. As demonstrated in [38], while full-attention mechanisms can be costly for very long sequence predictions, they are more efficient than other attention-variant models for moderate lengths, maintaining the lowest running time until output lengths exceed 2048. This is much larger than the typical settings for system load predictions that we focus on. The conclusion is also supported by the computational efficiency experiments in §5.5.

5 Experiments

Datasets. To validate MetaEformer across diverse system scenarios, we collect datasets from power grids, cloud platforms, and traffic systems. However, these datasets vary in their representation of complexity and dynamics. To magnify the challenges, we employ the Augmented Dickey-Fuller (ADF) test [8] to evaluate the datasets' non-stationarity, a key indicator of their dynamic nature. Higher ADF statistics and P-values signify greater variability. For multi-series datasets, ADF is computed per series and averaged. We apply a 99% confidence interval to determine non-stationarity. Complete ADF results are reported in Appendix A.3.

Based on the ADF results, we select three non-stationary time series from power datasets [24, 45]: ECL_{32, 57, 270}, originating from ECL. For cloud datasets, following [17], we opt for the ECW dataset, particularly with Switch and New APP datasets for only model testing, highlighting the model's efficacy with frequent changes and new entities. Furthermore, we enrich this scenario with the CBW (Cloud Bandwidth Workload) dataset, a network load collection from a commercial cloud platform, capturing real-world, highly dynamic scenarios. Among three traffic datasets [1, 25], we select the series with the highest ADF value, which demonstrate notable dynamic shifts in its latter half, as illustrated in Fig. 1.

Baselines. Fifteen baselines are compared, including six generic forecasting models: UniTime [12], PaiFilter [40], FreTS [41], TSMixer [7], Crossformer [44], and TimesNet [37]; four decomposition-based models: DLinear [42], Quatformer [9], FEDformer [46] and Autoformer [38]; three anti-non-stationary models: SAN [30], Koopa [28], and Non-stationary [29]; two clustering-based models: DynEformer [17] and VaRDE-L [39].

5.1 Implementation Details

All datasets have a sampling interval of 1 hour. The input load series length is $L = 48$, and the prediction length is $L_y = 24$, aiming to predict the load for the next day. As per the original study guidelines, dataset splits are as follows: the ECW dataset follows a 6:2:2 training, validation, and test ratio; the ECL dataset uses a 7:1:2 split, while the Traffic dataset is divided in an 8:1:1 ratio to capture end-of-dataset shifts without full disclosure. The CBW dataset, specifically collected for this research, adheres to an 8:1:1 division.

For the Meta-pattern Pooling mechanism, the STD decomposition period is fixed at 24, with a slice length $s = 16$. The MetaEformer's MPP updates every 50 batches, applying an update rate $\gamma = 0.1$. Key parameters, MPP size P and Echo's Top-K are detailed in §5.4. Models are trained with the ADAM [23] optimizer with an initial learning rate of 10^{-4}. Batch size is set to 256 and the start token length L_{token} is set to 12. All experiments are repeated three times. **Metrics**: MSE = $\frac{1}{n}\sum_{i=1}^{n}(Y-\hat{Y})^2$ and MAE = $\frac{1}{n}\sum_{i=1}^{n}|Y-\hat{Y}|$.

5.2 Main Results

The principal experiment results are reported in Table 2, from which we can observe that: **(1)** MetaEformer achieves consistent state-of-the-art performance compared to 15 baselines across 8 load benchmarks under 3 system scenarios. Specifically, against generic models, MetaEformer decreases overall MSE by 24.3%/ 19.5%/ 34.2% in cloud, power, and traffic scenarios, respectively; it surpasses *decomposition-based models* with MSE reductions of 38.3%/ 51.4%/ 24.8% and *anti-non-stationary models* with reductions of 45.6%/ 24.5%/ 24.4%. Last but not least, it outperforms *clustering-based models* by diminishing 51.8%/ 65.9%/ 63.4% MSE.

(2) MetaEformer exhibits superior adaptability in dynamic system scenarios, with its performance advantage increasing as system dynamics intensify. While the carefully selected dynamic datasets degrade baseline performance (e.g., most clustering-based and generic models see their MSE double from ECW to CBW), MetaEformer effectively mitigates this impact. Its lead over ECW's second-best model, DynEformer, expands from 1% to 20% on CBW.

(3) As state-of-the-art models under different classes, TimesNet, Dlinear, SAN etc. achieve sporadic best/second-best results in Table 2). However, these models' optimal performance is confined to specific scenario and lacks the generalizability that MetaEformer offers across various systems. Beyond accuracy, MetaEformer also provides the efficiency and interpretability essential for real-world industrial forecasting, which are absent in the baseline models.

5.3 Ablation Study and Model Interpretation

As shown in Table 3, we conduct a series of ablation studies on MetaEformer, with specific datasets for testing each system scenario: CBW-Cloud, ECL_270-Power, Traffic_852-Traffic.

Meta-pattern Pool. De-MPP deactivates the MPP component, meaning the meta-patterns are stochastically initialized and no longer updated. The MPP absence leads to an average ascend in MSE and MAE by 54% and 34%, respectively. This substantial improvement underscores the necessity of purifying and maintaining meta-patterns, which are crucial for capturing and adapting to the nuanced complexity and dynamics of time series data.

Thanks to the clustering-like paradigm, the Meta-pattern Pooling process, including MPP initialization and updates, is inherently interpretable and can be clearly visualized (detailed in Appendix A.5.1), helping operators understand common patterns of full system loads and the model's core focus during real-world deployment.

Table 2: Load forecasting performance under various system scenarios. A lower MSE or MAE indicates a better prediction.

Datasets	Cloud								Power						Traffic	
	CBW		ECW		New APP		Switch		ECL_32		ECL_57		ECL_270		Traffic_852	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
MetaEformer	**0.103**	**0.130**	**0.066**	**0.137**	**0.030**	0.118	**0.055**	0.155	**0.029**	0.130	**0.082**	**0.170**	0.042	0.149	**0.083**	**0.132**
UniTime	0.202	0.189	0.105	0.183	0.039	0.127	0.058	0.149	0.037	0.150	0.127	0.214	0.152	0.303	0.117	0.167
PaiFilter	0.179	0.172	0.079	0.139	0.032	**0.112**	0.059	0.153	0.035	0.136	0.103	0.193	**0.038**	**0.143**	0.099	0.173
FreTS	0.164	0.156	0.088	0.159	0.044	0.152	0.057	0.148	0.047	0.158	0.087	0.214	0.044	0.150	0.181	0.219
TSMixer	0.174	0.172	0.099	0.171	0.045	0.154	0.057	0.157	0.036	0.138	0.123	0.212	0.048	0.154	0.156	0.188
Crossformer	0.107	0.137	0.114	0.190	0.040	0.140	0.064	0.163	0.046	0.152	0.088	0.212	0.044	0.151	0.167	0.216
TimesNet	0.155	0.156	0.081	0.143	0.046	0.140	0.066	0.160	0.031	0.128	0.099	0.186	0.043	0.149	0.091	0.152
DLinear	0.114	0.131	0.123	0.202	0.039	0.133	0.067	0.162	0.062	0.199	0.136	0.226	0.068	0.198	0.098	0.177
Quatformer	0.212	0.202	0.192	0.272	0.054	0.158	0.059	0.143	0.168	0.329	0.160	0.263	0.077	0.207	0.106	0.200
FEDformer	0.249	0.237	0.160	0.257	0.047	0.152	0.056	**0.141**	0.184	0.346	0.126	0.231	0.060	0.181	0.107	0.198
Autoformer	0.164	0.193	0.079	0.146	0.111	0.229	0.158	0.270	0.166	0.320	0.141	0.248	0.064	0.188	0.138	0.241
SAN	0.176	0.182	0.178	0.266	25.913	3.696	1.688	0.815	0.049	**0.117**	0.116	0.209	0.044	0.150	0.098	0.171
Koopa	0.197	0.192	0.093	0.162	0.058	0.163	0.072	0.176	0.037	0.139	0.086	0.204	0.047	0.156	0.166	0.199
Non-sta.	0.203	0.183	0.089	0.175	0.063	0.175	0.068	0.171	0.043	0.161	0.132	0.224	0.047	0.161	0.090	0.133
DynEformer	0.124	0.161	0.067	0.141	0.090	0.207	0.067	0.169	0.063	0.184	0.138	0.239	0.089	0.224	0.144	0.257
VaRDE-L	0.541	0.455	0.150	0.218	0.556	0.502	0.270	0.369	0.525	0.555	0.379	0.366	0.165	0.285	0.532	0.554

Echo Layer. De-EL replace the Echo Layer with a linear layer to directly fuse MPP to forecasting, resulting in a 33% increase in MSE and 21% in MAE. The Echo Layer's primary function is to adaptively leverage the MPP to reconstruct patterns from deconstructed loads, which is especially effective in tackling the concept drift and few-shots challenges. The results indicate its criticality in maintaining forecasting accuracy in the face of dynamic system behaviors. We further illustrate the Echo Layer in Appendix A.5.2, showing how it leverages the MPP to adaptively drive load forecasting.

Echo Padding and SI Embedding. Deactivating Echo Padding (De-EP) and reverting to zero-padding results in a moderate performance decline, with up to a 10% increase in MSE. This suggests that padding meta-patterns provides more informative context compared to placeholders, particularly in scenarios with frequent few-shot entity introductions (e.g., N. APP). Similarly, replacing the SI Embedding with a direct stacking approach (De-SI) increases MSE by 13%, spotlighting the utility of integrating temporal and static information via learnable linear layers.

5.4 Parameters Sensitivity

In this section, we explore the sensitivity of two key parameters, P and K, through experiments on diverse domain datasets. P determines the capacity of MPP to sustain meta-patterns, significantly impacting performance and outcomes in various system contexts. As shown in Fig. 5(a), MAE decreases and then increases as P grows from 50 to 850, with the optimal P differing by scenario. We observe a positive correlation between this optimal value and the scenario's dynamism. Specifically, the most dynamic cloud scenarios exhibit the largest optimal P values (650 for CBW and ECW), while the relatively stable traffic scenarios have the smallest P value (350 for Traffic). K denotes the count of relevant meta-patterns selected from the MPP per Echo. Fig. 5(b) indicates MAE stability for K within 10 to 330. We attribute this stability to the adaptive fusion weights within the Echo, which further amplify or filter effects, making the Echo's impact relatively stable over a wide K range.

Table 3: Ablation of MetaEformer in various datasets. The 'Promo.' indicate the average enhancement in MSE and MAE.

Dataset		CBW	E._270	T._852	Switch	N. APP	Promo.
MetaE.	MSE	0.103	0.042	0.083	0.056	0.030	-
	MAE	0.130	0.149	0.132	0.155	0.118	
De-MPP	MSE	0.293	0.096	0.152	0.096	0.042	**54%**
	MAE	0.262	0.211	0.174	0.235	0.149	**34%**
De-EL	MSE	0.228	0.046	0.093	0.065	0.033	**33%**
	MAE	0.257	0.159	0.147	0.180	0.127	**21%**
De-EP	MSE	0.120	0.050	0.093	0.056	0.031	**10%**
	MAE	0.157	0.167	0.146	0.156	0.120	**8%**
De-SI	MSE	0.123	w/o static		0.060	0.035	**8%**
	MAE	0.214			0.174	0.123	**13%**

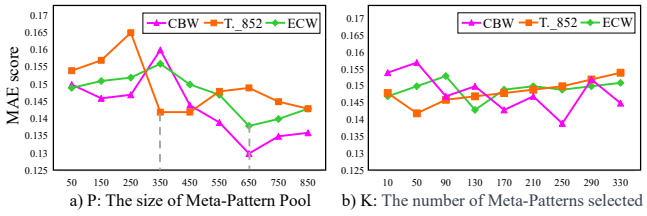


Figure 5: Performance of MetaEformer at different MPP size P and number K of meta-patterns selected by Echo Layer.

5.5 Computational Efficiency

Fig. 6 shows the average training time per epoch and the corresponding prediction performance (MSE) for different models on different datasets. MetaEformer shows lower training times with top prediction accuracy, highlighting its superior efficiency and performance. This supports the analysis in §4.3, where the added modules in MetaEformer have negligible time complexity compared to the Transformer. Compared to other Transformer-based models, MetaEformer exhibits competitive time efficiency and the highest predictive performance on both datasets (ECL_57 and Traffic_852). This means that MetaEformer can meet the demands of most practical applications, even those with stringent time requirements.

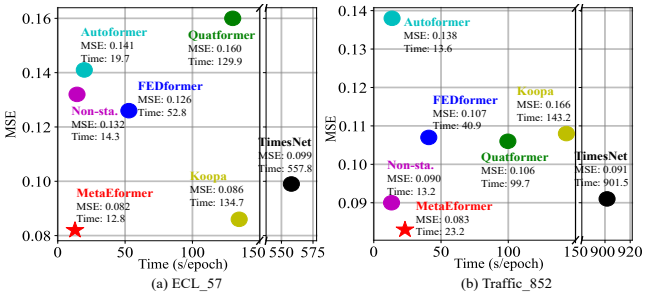


Figure 6: Efficiency of Transformer-based models.

6 Conclusion

This paper presents MetaEformer, a model built on the novel paradigm of meta-patterns, designed to forecast complex and dynamic loads in various real-world systems, addressing the challenges of complex patterns, concept drift and few-shot. MetaEformer introduces a novel Meta-pattern Pooling mechanism for purifying representative meta-patterns and an Echo mechanism for adaptively deconstructing and reconstructing load patterns. Experiments demonstrate that MetaEformer outperforms fifteen state-of-the-art baselines across diverse datasets, while offering end-to-end efficiency and clear interpretability, making it particularly well-suited for system load forecasting tasks in complex and dynamic environments.

Acknowledgments

We thank the anonymous reviewers and the area chair for their insightful comments. This work is supported by the Beijing-Tianjin-Hebei Basic Research Cooperation Program (F2024201070), the Key Program of the National Natural Science Foundation of China (U23B2049), the Tianjin Natural Science Foundation (23JCYBJC00780), the Tianjin Xinchuang Haihe Lab (22HHXCJC00002), and the National Natural Science Foundation of China (62472311).

References

[1] Chao Chen, Karl Petty, Alexander Skabardonis, Pravin Varaiya, and Zhanfeng Jia. 2001. Freeway performance measurement system: mining loop detector data. *Transportation Research Record* 1748, 1 (2001), 96–102.

[2] Mouxiang Chen, Lefei Shen, Han Fu, Zhuo Li, Jianling Sun, and Chenghao Liu. 2024. Calibration of Time-Series Forecasting: Detecting and Adapting Context-Driven Distribution Shift. *ACM SIGKDD* (2024), 341–352.

[3] Guillaume Chevillon. 2007. Direct multi-step estimation and forecasting. *Journal of Economic Surveys* 21, 4 (2007), 746–785.

[4] Robert B Cleveland, William S Cleveland, Jean E McRae, and Irma Terpenning. 1990. STL: A seasonal-trend decomposition. *J. Off. Stat* 6, 1 (1990), 3–73.

[5] Jovani Dalzochio, Rafael Kunst, Edison Pignaton, Alecio Binotto, Srijnan Sanyal, Jose Favilla, and Jorge Barbosa. 2020. Machine learning and reasoning for predictive maintenance in Industry 4.0: Current status and challenges. *Computers in Industry* 123 (2020), 103298.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*.

[7] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 459–469.

[8] Graham Elliott, Thomas J Rothenberg, and James H Stock. 1992. Efficient tests for an autoregressive unit root.

[9] Chen et al. 2022. Learning to Rotate: Quaternion Transformer for Complicated Periodical Time Series Forecasting. In *Proceedings of the 28th ACM SIGKDD (KDD '22)*. 146–156.

[10] Hugo et al. 2023. An embedding-based non-stationary fuzzy time series method for multiple output high-dimensional multivariate time series forecasting in IoT applications. *Neural Computing and Applications* 35 (2023), 9407–9420.

[11] Lukas et al. 2022. Few-shot forecasting of time-series with heterogeneous channels. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 3–18.

[12] Liu et al. 2024. UniTime: A Language-Empowered Unified Model for Cross-Domain Time Series Forecasting. In *ACM WWW 2024*.

[13] Wei Fan, Pengyang Wang, Dongkun Wang, Dongjie Wang, Yuanchun Zhou, and Yanjie Fu. 2023. Dish-TS: A General Paradigm for Alleviating Distribution Shift in Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 7522–7529.

[14] Alberto Gasparin, Slobodan Lukovic, and Cesare Alippi. 2022. Deep learning for time series forecasting: The electric load case. *CAAI Transactions on Intelligence Technology* 7, 1 (2022), 1–25.

[15] Xiao Han, Xiangyu Zhao, Liang Zhang, and Wanyu Wang. 2023. Mitigating Action Hysteresis in Traffic Signal Control with Traffic Predictive Reinforcement Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. 673–684.

[16] Rui Hu, Jie Chen, and Li Zhou. 2022. A transformer-based deep neural network for arrhythmia detection using continuous ECG signals. *Computers in biology and medicine* 144 (2022), 105325.

[17] Shaoyuan Huang, Zheng Wang, Heng Zhang, Xiaofei Wang, Cheng Zhang, and Wenyu Wang. 2023. One for All: Unified Workload Prediction for Dynamic Multi-Tenant Edge Cloud Platforms. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. 788–797.

[18] Shaoyuan Huang, Heng Zhang, Xiaofei Wang, Min Chen, Jianxin Li, and Victor C. M. Leung. 2024. Fine-Grained Spatio-Temporal Distribution Prediction of Mobile Content Delivery in 5G Ultra-Dense Networks. *IEEE TMC* 23, 1 (2024).

[19] Vinodh Kumaran Jayakumar, Jaewoo Lee, In Kee Kim, and Wei Wang. 2020. A Self-Optimized Generic Workload Prediction Framework for Cloud Computing. In *2020 IEEE IPDPS*. IEEE, 779–788.

[20] Yilun Jin, Kai Chen, and Qiang Yang. 2022. Selective Cross-City Transfer Learning for Traffic Prediction via Source City Region Re-Weighting. In *Proceedings of the 28th ACM SIGKDD Conference (KDD '22)*. 731–741.

[21] In Kee Kim, Wei Wang, Yanjun Qi, and Marty Humphrey. 2018. CloudInsight: Utilizing a Council of Experts to Predict Future Cloud Application Workloads. In *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*. 41–48.

[22] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2021. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In *ICLR*.

[23] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[24] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyou Zhou, Wenhu Chen, Yu-Xiang Wang, and Xifeng Yan. 2019. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Neurips* 32 (2019).

[25] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations (ICLR)*.

[26] Yanan Li, Haitao Yuan, Zhe Fu, Xiao Ma, Mengwei Xu, and Shangguang Wang. 2023. ELASTIC: Edge Workload Forecasting Based on Collaborative Cloud-Edge Deep Learning. In *ACM WWW 2023 (WWW '23)*. 3056–3066.

[27] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. 2021. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* 37, 4 (2021), 1748–1764.

[28] Yong Liu, Chenyu Li, Jianmin Wang, and Mingsheng Long. 2023. Koopa: Learning Non-stationary Time Series Dynamics with Koopman Predictors. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*.

[29] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in*

Neural Information Processing Systems (NeurIPS) 35 (2022), 9881–9893.
[30] Zhiding Liu, Mingyue Cheng, Zhi Li, Zhenya Huang, Qi Liu, Yanhu Xie, and Enhong Chen. 2023. Adaptive Normalization for Non-stationary Time Series Forecasting: A Temporal Slice Perspective. In *Thirty-seventh NeurIPS*.
[31] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. 2019. Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 31, 12 (2019), 2346–2363.
[32] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.
[33] M Sifuzzaman, M Rafiq Islam, and Mostafa Z Ali. 2009. Application of wavelet transform and its advantages compared to Fourier transform. (2009).
[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
[35] Qingsong Wen, Linxiao Yang, Tian Zhou, and Liang Sun. 2022. Robust Time Series Analysis and Applications: An Industrial Perspective. In *Proceedings of the 28th ACM SIGKDD (KDD '22)*. 4836–4837.
[36] Gerald Woo, Chenghao Liu, Doyen Sahoo, Akshat Kumar, and Steven Hoi. 2022. Etsformer: Exponential smoothing transformers for time-series forecasting. *arXiv preprint arXiv:2202.01381* (2022).
[37] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022).
[38] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS* 34 (2021), 22419–22430.
[39] Eunju Yang and Chan Hyun Youn. 2023. Temporal Data Pooling With Meta-Initialization for Individual Short-Term Load Forecasting. *IEEE Transactions on Smart Grid* 14, 4 (2023), 3246–3258.
[40] Kun Yi, Jingru Fei, Qi Zhang, Hui He, Shufeng Hao, Defu Lian, and Wei Fan. 2024. FilterNet: Harnessing Frequency Filters for Time Series Forecasting. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
[41] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. 2024. Frequency-domain MLPs are more effective learners in time series forecasting. *Neurips* 36 (2024).
[42] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
[43] Yuchen Zhang, Mingsheng Long, Kaiyuan Chen, Lanxiang Xing, Ronghua Jin, Michael I Jordan, and Jianmin Wang. 2023. Skilful nowcasting of extreme precipitation with NowcastNet. *Nature* 619, 7970 (2023), 526–532.
[44] Yunhao Zhang and Junchi Yan. 2023. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *The eleventh international conference on learning representations*.
[45] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *AAAI 2021* 12B (2021), 11106–11115.
[46] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *ICML*. PMLR, 27268–27286.

A Appendix

A.1 Meta-pattern Pool Update Algorithm

The process of meta-pattern pool update is shown in Algorithm 2.

Algorithm 2 Meta-pattern Pool Update

Input: the input series X, waveform length s, threshold τ, update rate γ
Output: Updated Meta-pattern pool $\mathcal{P}$

Extract waveforms $W = \text{Slice}(\text{STD}(X))$
Calculate similarity matrix SM between W and $\mathcal{P}$
Find max similarity and corresponding index $(\mathcal{V}_{\max}, \mathcal{I}_{\max}) = \text{Max}(SM, \text{axis} = 1)$
Filter rows with similarity above threshold $\mathbf{U} \leftarrow \mathcal{V}_{\max} > \tau$
$\mathcal{P}[\mathcal{I}_{\max}[\mathbf{U}]] = (1-\gamma)\mathcal{P}[\mathcal{I}_{\max}[\mathbf{U}]] + \gamma W[\mathbf{U}]$
for idx in $\{i | \mathbf{U}[i] = \text{false}\}$ **do**
    /* $\mathcal{P}$ has been filled. */
    if $\neg\exists$ row r in $\mathcal{P} : r = \mathbf{0}$ **then**
        $\mathcal{P}[\mathcal{I}_{\max}[idx]] = (1-\gamma)\mathcal{P}[\mathcal{I}_{\max}[idx]] + \gamma W[idx]$
    else
        $\mathcal{P}[r] = W[idx]$
    end if
end for
return $\mathcal{P}$

A.2 Computational Complexity Analysis

This section analyzes the computational complexity of MetaEformer. As a Transformer-based model, MetaEformer includes the basic computational units of the Transformer. Therefore, we first outline the complexity of the Transformer, followed by the additional complexity introduced by the new modules in MetaEformer.

Transformer Complexity. For an input length L, batch size B, and hidden layer dimension d, the computational complexity of a Transformer primarily arises from two sources, **Self-attention:** $O(L^2d)$ and **Feed-forward layers:** $O(Ld^2)$. If the Transformer has N layers, the total complexity is:

$$O(BN(L^2d + Ld^2)) \tag{16}$$

MetaEformer's Modules. Assuming a slice length s, each sequence can be divided into $L/s = n$ slices (usually small and can be neglected), and the size of the MPP is P. The additional complexities introduced by MetaEformer can be categorized as follows:

(1) **MPP Construction.** The complexity of MPP construction includes three components: (1) Decomposition and slicing, both with time complexity $O(BL)$; (2) Similarity calculation, with complexity $O\left(\left(\frac{BL}{s}\right)^2 \cdot s\right) = O(B^2s)$; and (3) The operations of finding maximum similarity, checking thresholds, and generating the weighted MPP, which incur a complexity of $O(BN)$. Among these, the similarity calculation is the dominant factor, resulting in an overall complexity of $O(B^2s)$.
(2) **MPP Updating.** The primary computational cost of MPP updating lies in the similarity calculation, which has a time complexity of $O\left(B \cdot \frac{L}{s} \cdot P \cdot s\right) = O(BLP)$.
(3) **Echo Layer.** The complexity of the Echo Layer is mainly determined by two operations: (1) Top-K similarity calculation, with complexity $O\left(n \cdot B \cdot \frac{L}{s} \cdot P \cdot s\right) \approx O(BLP)$; and (2) The Softmax and concatenation steps, with complexity $O(BsK)$.

Since the computational modules are executed serially, the total new complexity introduced by MetaEformer is $O(B^2s) + O(BLP) + O(BLP) + O(BsK) \approx O(B(Bs + LP + sK))$. Note that MetaEformer does not perform MPP construction and updating at every training iteration. Specifically, MPP construction only occurs during the first training batch, while updating frequency is generally set to $\frac{1}{M}$ of training iterations. This reduces the added complexity to:

$$O(B(\frac{Bs}{E} + \frac{LP}{M} + sK))$$

where E is the total number of iterations required for training (epoch*batches per epoch), M is the frequency of performing MPP updates ($M = 50$ in our setup). Ultimately, the complexity of the native components of Transformer exceeds the new components

Table 4: ADF test results on various system load datasets. Higher Statistic and P-value indicate more pronounced dynamic characteristics.

Scenario	Dataset	ADF Statistic	P-Value	Is_Stationary
Power	ETTm2	-4.0335	0.0012	T
	ETTm1	-3.8811	0.0022	T
	ETTh1	-3.4880	0.0083	T
	ETTh2	-3.5971	0.0058	T
	ECL	-8.4449	0.0051	6 F, 315 T
	ECL_32	-2.4347	0.1322	F
	ECL_57	-1.9932	0.2896	F
	ECL_270	-2.8343	0.0535	F
Cloud	**CBW**	-4.2933	0.1536	354 F, 476T
	ECW	-4.8440	0.0680	180 F, 617 T
	Switch	-3.3600	0.0490	1 F, 2 T
	New APP	-2.3176	0.2720	7 F, 4 T
Traffic	Traffic	-15.0210	0.0000	862 T
	Traffic_852	-5.0309	0.0000	T
	PEMS_BAY	-26.3736	0.0000	325 T
	METR_LA	-16.0138	0.0000	207 T

introduced by MetaEformer by an order of:

$$\frac{N(Ld + d^2)}{K}$$

with common deep learning task settings, this scale can be hundreds of times large. Therefore, we assert that the additional computational complexity from MetaEformer is **negligible** and ignorable.

A.3 ADF Test Results

In Table 4, we present the complete ADF test results on the power, cloud, and traffic datasets. The bolded results confirm the datasets' suitability for our experimental needs.

A.4 Prediction Showcases

To assess model performance on dynamic behaviors, we compare MetaEformer with four state-of-the-art models on three dynamic datasets, as shown in Fig. 8. The Switch and New App datasets, configured according to [17], are fully independent of the training and are used to test the models' abilities to handle concept-drift (switch behavior) and few-shot learning (new entity addition).

In the switch dataset (Fig. 8 (a)), even when high workload after the switch are provided, most models are still affected by the low workload from the previous period in the lookback window. This results in predictions much lower than the actual values. MetaEformer, using the MPP, relies more on similar meta-patterns for prediction, effectively handling switch behavior. In the new app dataset (Fig. 8 (b)), most models struggle to achieve fine-grained predictions due to the limited few-shot data, leading to inaccuracies in predicting extreme values, which are often crucial. The proposed Echo mechanism, by calculating similarity using peaks and troughs, effectively merges meta-patterns from accumulated similar cases, making it particularly adept at capturing these extreme values.

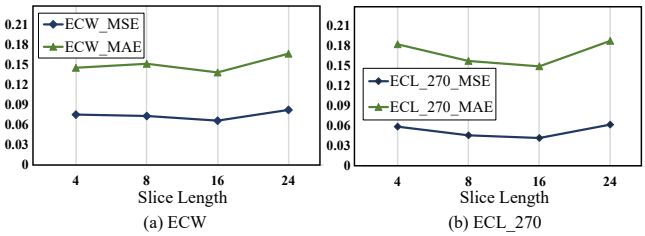


Figure 7: Prediction performance with varying slice lengths.

A.5 Key Mechanisms Interpretation

A.5.1 Meta-pattern Pooling. To explain the Meta-pattern Pooling mechanism, we visualize the initial construction and five subsequent updates of the MPP during MetaEformer's training on the CBW dataset using heatmaps. Each update corresponds to the full batch of data input for the current iteration. In the heatmaps, the vertical axis represents the 650 meta-patterns in the MPP (which may not be fully filled initially), and the horizontal axis shows the original series for the initial construction and the changes in each meta-pattern before and after updates. Darker areas indicate greater changes in meta-patterns. The final result is shown in Fig. 9.

From Fig. 9, it can be observed that in the early updates, due to the limited amount of data learned by the model, new meta-patterns are still being discovered and filled into the MPP during updates, as shown in the 1st updates. Significant changes occur when new patterns are introduced. The MPP quickly learns a richer and sufficient set of meta-patterns, resulting in no new meta-patterns being added. Instead, the focus shifts to updating the existing meta-patterns. This process demonstrates that the MPP update mechanism adeptly captures subtle variations in meta-patterns from subsequent input data and successfully purifies new, distinct meta-patterns.

A.5.2 Echo Layer. As illustrated in Fig. 10, each Echo operation in MetaEformer actively selects suitable meta-patterns for integration based on the current input waveform, driven by the proposed similarity calculation (Sim), as highlighted in the figure. The selected meta-patterns reflect underlying structures from the original waveform while enriching it with additional global pattern information. This complementary information is crucial for the Decoder, enhancing the accuracy and robustness of the final predictions.

Throughout the MetaEformer forecasting process, the Echo Layer functions analogously to a wavelet transform. It deconstruct the original input to identify the most relevant meta-patterns (basis functions) from the MPP and determines how these meta-patterns should be integrated into the current embedding space (reconstruct). By guiding the model on both what meta-patterns are most representative and how they should be fused, the Echo Layer provides complementary information that enhances MetaEformer's ability to generate precise predictions.

A.6 Sensitivity Analysis of Slice Length

To investigate the impact of slice lengths s, we perform MetaEformer on ECW and ECL_270 with varying $s = [4, 8, 16, 24]$. As observed in Fig. 7, the forecasting performance does not significantly vary with different values of s, indicating the robustness of the MetaEformer against the slice length hyperparameter.

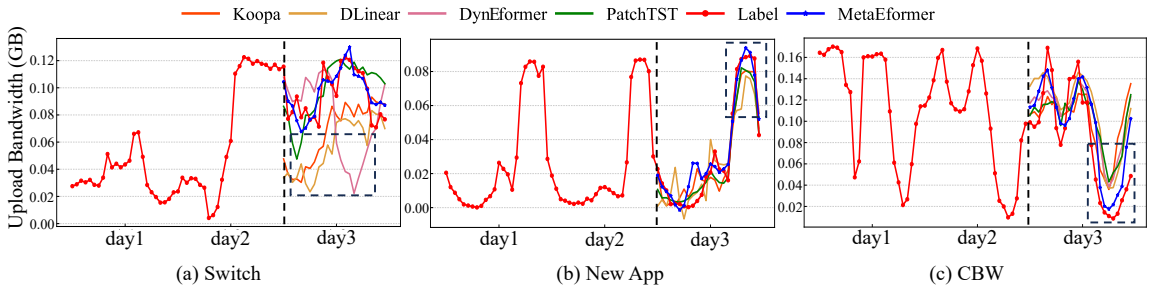


Figure 8: Prediction cases from the Switch, New App and CBW dataset.

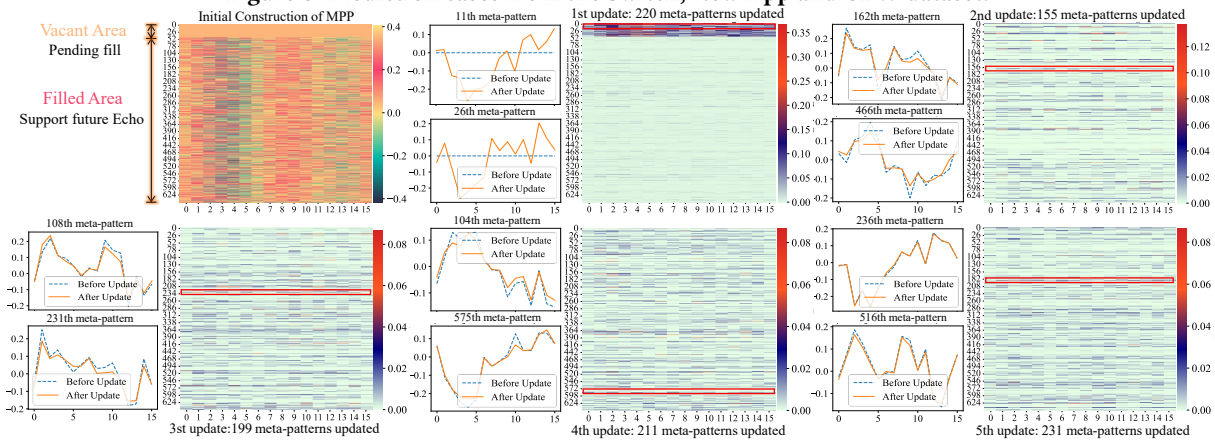


Figure 9: Visualization of MPP construction and updates. The first heatmap represents the original values of the MPP, and the subsequent heatmaps represent the difference between the updated MPP and the original. The meta-patterns with the most significant changes are visualized separately (indicated by red boxes).

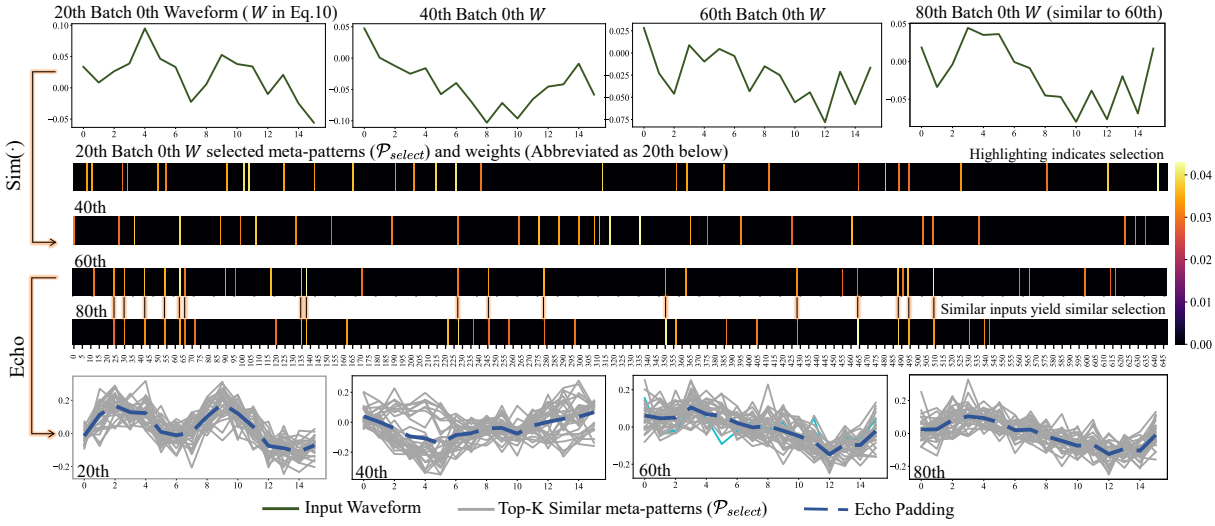


Figure 10: Displays the Echo mechanism for the input series (waveform) in every 20 batches during training on the CBW. The heatmaps depict the meta-patterns selection process within MPP, with non-selected meta-patterns set to zero. The bottom visualizes the Top-K meta-patterns $\mathcal{P}_{select}$ and the Echo Padding. Different waveforms echo meta-patterns with distinct fluctuation patterns, while similar waveforms tend to select the same meta-patterns (e.g., 60th and 80th).